

Outpainting Audio by Reusing What the Model Already Knows: A Minimal Full-Finetune Recipe for Latent Rectified-Flow Audio Diffusion

Francesco Colonnese
github.com/mrcolo/loop-training

Abstract

Audio outpainting — continuing a recording past its end — is usually framed as a new capability requiring new machinery: extra input channels, adapter branches, or a separately trained continuation head. We observe that a modern latent audio diffusion model already contains everything the task needs. Stable Audio 3 Medium is trained with inpainting conditioning, taking a binary mask and a masked latent as local-additive conditions, and its mask distribution already includes a causal mode that retains a random-length prefix. Outpainting is that mode. Consequently a finetune for outpainting can change no architecture at all: no new channels, no adapters, no surgery on the input projection. Only the transformer weights move.

The recipe that follows is complete, targets a single 24 GB consumer GPU, and trains all 1.453 B transformer parameters. Its central finding is about which published weights admit it. The model ships in two forms, a rectified-flow checkpoint and the adversarially post-trained checkpoint used for eight-step inference, and a squared-error finetune is valid only on the first. We show that the post-trained weights encode a sample predictor, that a thousand squared-error steps convert them into a conditional-mean predictor, and that the eight-step sampler cannot tolerate the difference. The conversion is invisible to every scalar in the training log — loss falls monotonically throughout — and is measurable in one forward pass by comparing the scale of the one-step estimate to its correlation with the truth. We give that diagnostic, the corrected recipe, and the memory budget that makes it fit.

1 Introduction

Generative audio models have converged on latent diffusion: a high-compression autoencoder maps waveforms to a short sequence of continuous vectors, and a diffusion transformer (DiT) is trained in that space. The recipe is now mature enough that the interesting questions have shifted from how to generate to how to control, and one of the most practically useful controls is continuation: given the first τ seconds of a recording, produce what plausibly follows.

The standard reflex is to treat this as an architec-

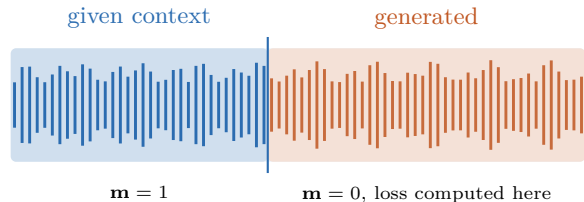


Figure 1: The task. A random-length prefix of a latent sequence is retained as context; the model generates the remainder. This is exactly the causal member of the mask family the base model was pretrained with, so no architectural change is required. The training loss is taken only over the generated region.

tural problem. Image inpainting models concatenate a mask and a masked image to the noisy latent, widening the input projection from C to $2C+1$ channels and zero-initialising the new columns. The same construction transfers to audio and works. But it is unnecessary when the base model was already trained with masked conditioning.

Contributions. (i) We identify that outpainting is a subset of the inpainting conditioning already present in Stable Audio 3, and show that restricting the pre-trained mask distribution toward its causal mode is sufficient to specialise the model for continuation (§5.1). (ii) We show that squared-error training on an adversarially post-trained checkpoint silently converts a sample predictor into a conditional-mean predictor, give a one-forward-pass diagnostic that detects it, and report the failure it produces in full (§4). (iii) We argue that the optimiser is part of the recipe rather than an implementation detail, and that an orthogonalised update is what makes a single learning rate valid across a transformer’s matrices (§7). (iv) We give a memory budget under which all 1.453 B transformer parameters, an orthogonalising optimiser and a weight average fit on one 24 GB GPU (§8), and a data path with no manifest and no preprocessing stage (§6). (v) We argue that per-step training loss is the wrong instrument for this objective and propose a frozen evaluation slice (§10) — while showing that even a well-behaved validation curve is not sufficient.

2 Related Work

Latent audio diffusion. Encoding waveforms into a compact latent sequence before diffusion is now standard, and the compression ratio has risen sharply: the autoencoder we build on maps 4096 waveform samples to a single 256-dimensional latent frame, a 10.77 Hz representation of 44.1 kHz stereo. High compression is what makes minute-scale generation tractable, and it is also what makes online encoding expensive (§9).

Masked generative modelling. Inpainting by channel-concatenating a mask and a masked signal originates in image diffusion and has been carried into audio. The variant we exploit differs in that the masked conditioning is injected as a local additive term rather than by widening the input projection, and — critically — is present at pretraining rather than bolted on afterwards.

Flow matching and rectified flow. Rectified flow replaces the variance-preserving diffusion path with a straight interpolation between data and noise, and predicts the constant velocity along it. Its simplicity and its compatibility with few-step distilled samplers have made it the default for recent audio models.

Full versus parameter-efficient finetuning. LoRA and adapters dominate finetuning practice because they fit on small hardware. We take the opposite position for this task: when the target domain differs from the pretraining distribution in timbre and production style rather than in semantics, the low-rank constraint is a real limitation, and a 1.45 B model can be fully finetuned on a consumer card given the right precision and optimiser-state choices.

3 Preliminaries

3.1 Latent representation

Let $\mathbf{x} \in \mathbb{R}^{2 \times N}$ be stereo audio at 44.1 kHz. A frozen autoencoder $\mathcal{E}$ maps it to

$$\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{256 \times T}, \quad T = \lceil N/4096 \rceil, \quad (1)$$

a 256-channel sequence at 10.77 frames per second. The encoder is a patched transformer: waveform samples are first grouped into patches of 256, then a 12-layer transformer with stride 16 produces the latent. The decoder mirrors it. Together they account for 0.852 B parameters and remain frozen throughout.

3.2 Rectified-flow objective

The generative model is a DiT v_θ trained with a rectified-flow (rf_denoiser) objective. For $t \in [0, 1]$ and $\epsilon \sim$

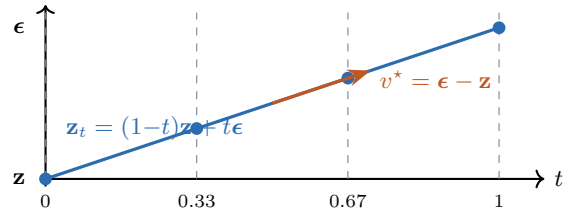


Figure 2: Rectified flow. Data and noise are joined by a straight line, and the network regresses the constant velocity along it. Because the path is linear, the velocity target does not depend on t — but the difficulty of predicting it does, which is the source of the variance discussed in §10.

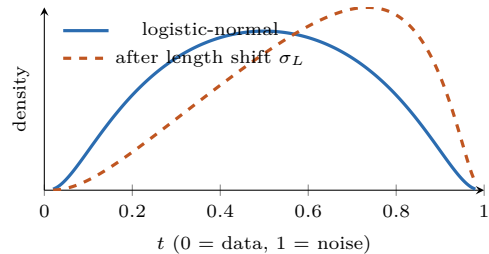


Figure 3: Timestep density. Mass concentrates in the middle of the schedule where the prediction problem is hardest, and the length-dependent shift moves it toward higher noise as the sequence grows. We inherit both from pretraining rather than substituting a uniform sampler.

$$\mathcal{N}(0, I),$$

$$\mathbf{z}_t = (1 - t) \mathbf{z} + t \epsilon, \quad (2)$$

$$v^* = \epsilon - \mathbf{z}, \quad (3)$$

so that $t=0$ is data and $t=1$ is noise, and the target velocity is constant along each path. The network is conditioned on t , on a text embedding, and on a scalar duration embedding.

3.3 Timestep sampling

Timesteps are drawn from a truncated logistic-normal distribution, reflected, and then warped by a sequence-length-dependent shift σ_L that moves probability mass toward higher noise for longer sequences:

$$t \sim \sigma_L(1 - \text{TLN}(0.075, 1)). \quad (4)$$

We reuse the pretraining sampler unchanged. Matching it matters: a finetune that samples timesteps differently from pretraining spends its gradient budget on a different part of the schedule than the one the model was optimised for.

t	Released		+ 1000 MSE steps	
	std ratio	ρ	std ratio	ρ
0.95	0.832	0.321	0.551	0.502
0.90	0.896	0.565	0.686	0.662
0.80	0.964	0.719	0.784	0.785
0.60	0.999	0.872	0.892	0.899
0.40	1.004	0.946	0.948	0.954
0.20	1.001	0.985	0.983	0.986

Table 1: Scale of the one-step estimate $\hat{\mathbf{z}}_0$ relative to the truth, against its correlation with the truth, on a real excerpt under a full mask. The released checkpoint emits a full-scale sample where it knows almost nothing. After a thousand squared-error steps the two columns coincide at every noise level, which is the defining signature of a conditional-mean predictor.

4 Which checkpoint admits a squared-error finetune

Stability publishes two weights for this model, and the choice between them is the single decision that determines whether the finetune works at all.

Base is the rectified-flow model: it predicts a velocity, and it is sampled with an ODE solver in roughly 50 steps with classifier-free guidance. Released is that model after adversarial post-training. It predicts a sample rather than a conditional mean, and it is sampled with ping-pong in 8 steps at guidance 1:

$$\hat{\mathbf{z}}_0 = \mathbf{z}_t - t v_\theta(\mathbf{z}_t, t, c), \quad (5)$$

$$\mathbf{z}_{t'} = (1 - t') \hat{\mathbf{z}}_0 + t' \mathbf{m}\epsilon', \quad \mathbf{m}\epsilon' \sim \mathcal{N}(0, I). \quad (6)$$

Equation (6) discards the previous iterate entirely and rebuilds it from $\hat{\mathbf{z}}_0$ and fresh noise. Every step therefore requires $\hat{\mathbf{z}}_0$ to be a full-scale, self-consistent clip on its own, at every noise level.

A squared-error objective does not produce that. Its minimiser is $\mathbb{E}[\mathbf{z}_0 \mid \mathbf{z}_t]$, whose scale shrinks exactly in proportion to how much the model actually knows: if ρ is the correlation between the estimate and the truth, then $\text{std}(\hat{\mathbf{z}}_0)/\text{std}(\mathbf{z}_0) \rightarrow \rho$. A sample predictor instead holds that ratio near 1 even where ρ is small. The two are distinguishable in a single forward pass.

Table 1 runs that test. A thousand squared-error steps convert the released checkpoint into a mean predictor: the two columns become equal throughout. At the first ping-pong step the generated signal has lost a third of its amplitude while (6) continues to inject noise at full scale. Perceptually the output flattens and hisses.

Two things make this failure difficult to catch. First, the correlation improves, $0.321 \rightarrow 0.502$ at $t = 0.95$: the model is getting better at the objective it was given, and validation loss falls monotonically throughout. Second, no scalar in the training log distinguishes the two

	$t=0.95$	$t=0.9$	$t=0.8$	$t=0.6$
Base	0.535/0.533	0.605/0.605	0.735/0.740	0.869/0.873
Base + Δ	0.578/0.558	0.639/0.626	0.757/0.753	0.879/0.877
Released	0.920/0.393	0.937/0.483	0.971/0.658	0.999/0.842
Released + Δ	0.965/0.408	0.952/0.506	0.967/0.681	0.995/0.849

Table 2: Amplitude / correlation of the one-step estimate, at 3000 finetuning steps. Equal entries denote a conditional-mean predictor. The transplanted model retains the separation that few-step sampling depends on.

regimes; the diagnostic has to be constructed deliberately.

The mechanism is not incidental — it is the stated purpose of the post-training stage, which works “by supplanting the MSE-based conditional mean loss ... with an adversarial loss”. Running squared error on those weights re-imposes the loss the stage was run to remove, and no learning rate makes that objective correct; a smaller one only slows the regression. Stability’s own trainer refuses the released checkpoint outright, and their documentation gives the remedy: adapters “are trained on the base checkpoint. Once trained, they can be applied to the post-trained model and will work as expected.” We therefore train Base, store checkpoints as the delta from its weights, and add that delta to Released at inference. The same rule is established in images: a distilled model is “impossible to train on directly because every step you train breaks down the compression more and more”.

4.1 The delta transplants

Training Base is only useful if the result can be moved onto Released, and the guidance that says it can is written for low-rank adapters, which are small by construction. A full finetune is not. Fetching one transformer layer of Released by byte range and comparing it with Base puts a number on the margin: adversarial post-training moved that layer’s weights by 1.38% on median, and after 1000 steps our update is already $0.85\times$ that, with two tensors past it. The question is therefore empirical.

Table 2 answers it with the same diagnostic, run on four models under identical noise. Base sits exactly on the conditional-mean line, 0.535 against 0.533, and stays there once our delta is added, which is correct: that is the objective it was trained with. Released separates sharply, and keeps separating after the delta is added: 0.965 against 0.408, against 0.920 and 0.393 for the untouched weights. The few-step behaviour is intact. Its correlation with the truth also rises, so the finetune’s knowledge of the target domain transfers into the accelerated model rather than merely surviving in it.

The practical consequence is that the two checkpoints divide labour cleanly: Base is what the objective is valid

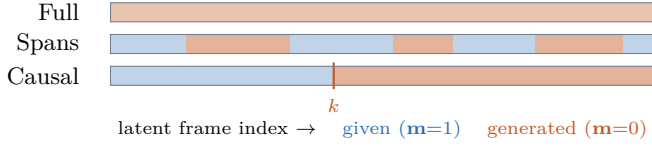


Figure 4: The pretrained mask family. Stable Audio 3 is pretrained with mask-type probabilities $[0.1, 0.8, 0.1]$ over [Spans, Full, Causal]. Outpainting is already in the support of this distribution — it is simply rare. Our finetune reweights to $[0, 0.1, 0.9]$.

against, Released is what inference runs on, and the finetune is transported between them as a float16 delta. Nothing about the recipe needs to change to get both.

5 Method

5.1 Outpainting as a restriction of pretrained masking

The base model consumes two extra conditioning tensors alongside the noisy latent: a binary keep-mask $\mathbf{m} \in \{0, 1\}^{1 \times T}$ and the masked latent $\mathbf{z} \odot \mathbf{m}$. These enter the transformer as a local additive term of dimension $256+1$, added to the token embeddings rather than concatenated to the input channels.

During pretraining, $\mathbf{m}$ is drawn from a family of mask types. Three matter here:

Full $\mathbf{m} = 0$ everywhere. Pure unconditional generation.

Spans a set of non-overlapping interior spans is zeroed. General inpainting.

Causal a prefix length $k \sim \mathcal{U}\{0, \dots, T\}$ is sampled and $\mathbf{m}_i = \mathbf{1}[i < k]$. This is outpainting.

Because Causal is already in the support of the pre-training distribution, the finetune is a reweighting rather than an extension. We set the mask-type probabilities to $[0, 0.1, 0.9]$, concentrating gradient on continuation while retaining a tenth of each batch at Full. That residual matters: training exclusively on causal masks steadily degrades the model’s unconditional behaviour, which is also the behaviour the Full branch of classifier-free guidance relies on at sampling time.

5.2 Two-term objective

The report is explicit that the loss is not taken on the generated region alone: it “is split into two independently averaged terms: a generation loss over the inpainted embeddings ($m = 0$) and a context preservation loss over the kept audio ($m = 1$)”, and the shipped configuration sets the weight on the second to 1.0. Writing $\bar{\mathbf{m}} = 1 - \mathbf{m}$

and $\varepsilon = v_\theta(\mathbf{z}_t, t, c, \mathbf{m}, \mathbf{z} \odot \mathbf{m}) - v^*$,

$$\mathcal{L}(\theta) = \underbrace{\frac{\|\varepsilon \odot \bar{\mathbf{m}}\|_2^2}{256 \sum_i \bar{\mathbf{m}}_i}}_{\text{generation}} + \underbrace{\frac{\|\varepsilon \odot \mathbf{m}\|_2^2}{256 \sum_i \mathbf{m}_i}}_{\text{context}}. \quad (7)$$

Both terms are averaged per sample before the batch mean. Pooling them into a single ratio instead would weight each region by its size, which down-weights precisely the long-context examples that outpainting is about. The context term is cheap — the model is handed the clean latent there, so the target is algebraically recoverable and the term sits near 10^{-2} — but it is what pins the model’s output on the prefix, which the sampler regenerates along with everything else.

5.3 Conditioning is constant, so compute it once

The model is conditioned on a text embedding from a T5Gemma encoder (256 tokens, 768-dimensional) and on a scalar duration embedding. For a single-domain finetune both the prompt and the excerpt length are fixed for the entire run, so the conditioning tensor is a constant. We evaluate the text encoder once at startup, cache the resulting tensor, and release the encoder’s 1.2 GB before the first training step. Classifier-free-guidance dropout is applied downstream by zeroing the cached tensor and therefore needs no encoder.

6 Data path

Conventional latent-diffusion training precomputes latents into a cache. Stability’s own configuration does this (pre_encoded: true). We deliberately do not, and the data path is correspondingly trivial: on each item we open the source file, seek to a uniformly random frame offset, and read exactly the excerpt we need.

This has three consequences worth stating. First, there is no preprocessing stage, no manifest, and no cache to invalidate when the excerpt length or sample rate changes. Second, memory is independent of corpus size — a 26.4-hour, 10.5 GB source file costs nothing to hold because it is never held. Third, the excerpt distribution is continuous rather than quantised to cache-entry boundaries, so the model sees a different alignment of musical structure on every epoch.

The cost is negligible: a random seek-and-decode of a 24s excerpt from a 10.5 GB FLAC takes 31 ms, and two dataloader workers hide it entirely behind GPU work.

7 The optimiser is part of the recipe

The shipped configuration asks for MuonAdamW: Muon on the attention and feed-forward matrices, AdamW on

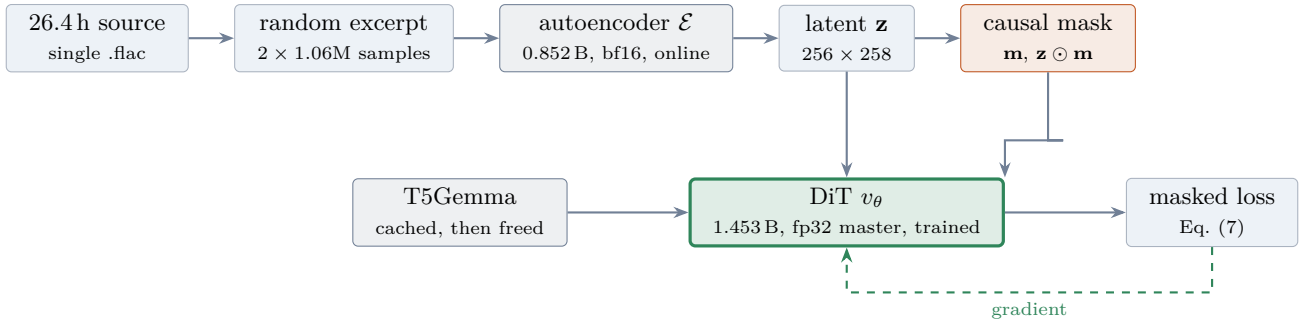


Figure 5: System overview. Grey blocks are frozen, green is trained. Nothing is precomputed: each step reads fresh audio off disk and encodes it in the same step it is used. The text encoder is evaluated once and then released, since its output is constant for a run.

Algorithm 1 One training step

- 1: $\mathbf{x} \leftarrow \text{RandomExcerpts}(\text{source}, B, \tau)$ $\triangleright$ 31 ms, on workers
 - 2: $\mathbf{z} \leftarrow \mathcal{E}(\mathbf{x})$ $\triangleright$ online, bf16, no grad
 - 3: $\mathbf{m} \leftarrow \text{CausalMask}(B, T; p_{\text{full}})$
 - 4: $t \leftarrow \sigma_L(1 - \text{TLN}())$
 - 5: $\epsilon \sim \mathcal{N}(0, I)$
 - 6: $\mathbf{z}_t \leftarrow (1 - t)\mathbf{z} + t\epsilon; \quad v^* \leftarrow \epsilon - \mathbf{z}$
 - 7: $\hat{v} \leftarrow v_\theta(\mathbf{z}_t, t, c, \mathbf{m}, \mathbf{z} \odot \mathbf{m})$ $\triangleright$ cfg dropout 0.1
 - 8: $\mathcal{L} \leftarrow \|(\hat{v} - v^*) \odot (1 - \mathbf{m})\|^2 / \sum(1 - \mathbf{m})$
 - 9: backward; clip to 1.0; AdamW8bit step
-

everything else. Substituting AdamW throughout is not a neutral simplification.

Muon orthogonalises the momentum buffer before applying it. Writing M for the buffer of a matrix parameter $W \in \mathbb{R}^{m \times n}$ and NS_5 for five steps of the quintic Newton-Schulz iteration,

$$W \leftarrow W - \eta \sqrt{\max(1, m/n)} \text{NS}_5(M + \mu G), \quad (8)$$

so the update has unit spectral norm by construction and η is the size of the step, in the same units, for every matrix in the network. Under AdamW the step size is set by the ratio of the gradient to its own running second moment, which varies by more than an order of magnitude across a transformer’s matrices. A single AdamW learning rate is therefore simultaneously too large for some matrices and too small for others, and the finetune degrades the model along the first group while failing to move it along the second. This is the mechanism behind a finetune whose loss falls while its samples get worse, and it compounds with the checkpoint error of §4.

Fused projections are split before orthogonalisation: a stacked qkv weight is several independent matrices and treating the stack as one is wrong. The configuration’s `fused_layer_patterns` field exists for exactly this. The momentum buffer is held in bfloat16, which halves its cost to 2.9 GB and is harmless because only the direction of (8) survives.

We also restore the exponential moving average the

configuration specifies, $\beta = 0.9995$ under a power-law warmup, and sample from it. It is kept in host memory and updated every eighth step, which costs a copy and returns 2.9 GB of device memory.

8 Fitting a full finetune on one GPU

Training all 1.453 B transformer parameters naively is out of reach: float32 weights, float32 gradients and float32 Adam moments alone require $1.453\text{B} \times 16\text{B} = 23.2\text{GB}$, before activations, before the autoencoder, and before the text encoder. Three choices bring it inside 24 GB.

1. Eight-bit Adam moments. The optimiser state drops from 11.6 GB to 2.9 GB. Master weights and gradients stay in float32, so the update itself is unaffected in precision where it matters.
2. The frozen autoencoder in bfloat16. It never receives gradients, so half precision costs nothing and saves 1.7 GB.
3. Gradient checkpointing throughout the transformer.

Releasing the text encoder after its single use recovers a further 1.2 GB. The resulting measured peak is 15.7 GiB, leaving comfortable headroom for the sampling and decoding done during periodic demos.

8.1 Checkpoint transport

The released checkpoint is 9.2 GB of float32 across 997 tensors (1.453 B transformer, 0.852 B autoencoder). Since training holds float32 master weights in memory regardless, storing bfloat16 on disk costs a single rounding at load. We therefore convert during download, streaming tensors in file order and writing a bfloat16 safetensors file directly, so the float32 artefact is never materialised. The stored model is 4.6 GB.

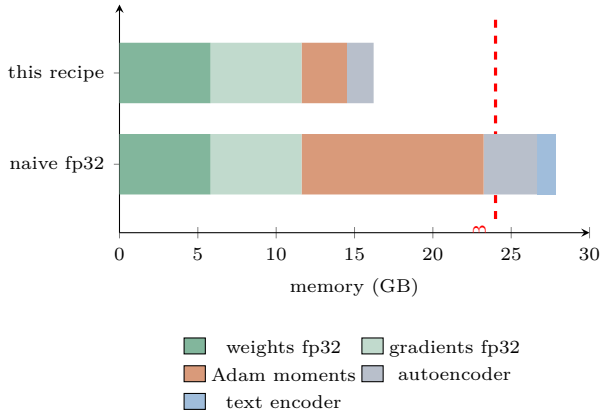


Figure 6: Memory budget. Eight-bit optimiser moments, a bfloat16 frozen autoencoder and releasing the text encoder move a 27.9 GB configuration to a measured 15.7 GiB peak. Activations under gradient checkpointing are small and not shown separately.

Stage	Time (s)
Excerpt read and decode (2 workers)	0.03
Autoencoder encode (online)	4.00
DiT forward + backward	0.45
Optimiser step	< 0.01
Total per step	4.70

Table 3: Measured per-step cost, RTX 3090, batch 4, 24s excerpts.

9 Cost structure

The autoencoder dominates, costing roughly nine times the transformer step it feeds. This is the honest price of online encoding, and it follows directly from the architecture: a 0.852 B encoder that must process 8085 patch tokens to emit 258 latent frames does substantially more work than a 1.453 B transformer operating on those 258 frames.

We note that this cost is a choice, not a constraint. The training loop consumes $\mathbf{z}$ and is indifferent to its provenance; substituting a precomputed latent cache recovers the 4s and changes nothing else. We prefer the online path for single-corpus finetunes, where setup cost dominates total cost.

10 Evaluation protocol

Per-step training loss is a single batch at a single randomly drawn timestep, and rectified-flow loss depends strongly on that timestep. Over any 500-step window of this run it spans roughly 0.75–0.92, which is wider than the total movement of the run. Reading a trend from it is guessing.

The frozen slice fixes everything that is not the model:

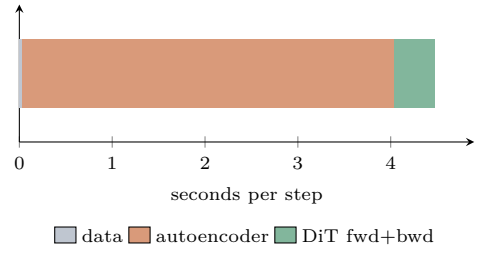


Figure 7: Where the step goes. Online encoding is 85% of wall clock. The trade is deliberate: it buys a pipeline with no preprocessing stage and no cache, at roughly 6× the per-step cost of training from precomputed latents.

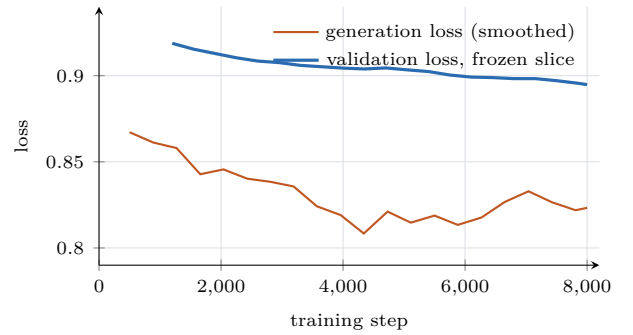


Figure 8: Both curves descend for the whole run. Neither indicates when to stop, and neither would have revealed the failure of §4. The frozen slice is the more legible of the two only because it holds excerpts, masks, timesteps and noise fixed; a single training batch at a random timestep spans 0.71–0.92 at any point in the run.

the same four excerpts, the same masks, the same four timesteps, the same noise. Successive values are then directly comparable, and Figure 8 shows it falling from 0.9275 to 0.8891 over 8000 steps.

The scalar is necessary but not sufficient. It descended smoothly through a configuration that was measurably destroying the model (§4), and it was still descending at the point where doubling the training bought nothing (§12). We therefore evaluate on audio: a fixed reference excerpt continued at every checkpoint, and periodically a grid of offsets drawn from an energy gate, crossed with noise seeds, with the models under comparison placed on identical trajectories.

That last condition is easy to get wrong. Ping-pong redraws noise inside its own loop, so seeding the initial latent leaves two models being compared on different paths; the measured gap between them moved by ten points of relative improvement once the global generator was seeded as well.

Step	Validation loss	Drop per 1000 steps
1	0.9275	—
1000	0.9195	0.0080
2000	0.9115	0.0080
4000	0.9095	0.0010
6000	0.9032	0.0032
8000	0.8891	0.0071

Table 4: The frozen slice descends monotonically over the whole run, including across the interval in which sample quality stopped improving (§12).

Initialisation	Stable Audio 3 Medium Base
Trained	DiT, 1.453 B, all parameters
Frozen	autoencoder 0.852 B, text encoder
Latent	256 ch @ 10.77 Hz (/4096)
Windows	47/95/190/380 s, sampled per step
Batch	1×4 accumulation
Objective	rectified flow, velocity target
Loss	generation + context, Eq. (7)
Mask types	0.55 full / 0.10 segments / 0.35 causal
Causal context	$\mathcal{U}(20, 60)$ seconds
Timesteps	truncated logit-normal, flipped, shifted
CFG dropout	0.1
Optimiser	Muon + AdamW (8-bit moments)
Muon	2×10^{-4} , momentum 0.95, 1.36 B params
AdamW	10^{-5} , (0.9, 0.95), wd 0.01, 94 M params
Schedule	200-step warmup, $\gamma = 10^6$, power 0.5
EMA	0.9995, power-law warmup
Grad clip	1.0
Precision	fp32 master, bf16 autocast
Sampling	Euler 50 steps, CFG 4 (Base) ping-pong 8 steps, CFG 1 (Released)
Peak memory	20.2 GiB
Throughput	4.4 s/step (RTX 3090)

Table 5: Complete hyperparameter set.

11 Recipe summary

Table 5 collects every value needed to reproduce the method. Four entries differ from the base model’s own pretraining configuration: the mask-type probabilities, reweighted toward Causal; the causal context length, drawn in seconds rather than as a fraction of the window; the learning rates, a fifth of the pretraining values because this is a finetune of a converged model; and eight-bit moments for the AdamW group. Everything else — the objective, the timestep sampler, the distribution shift, the guidance dropout rate, the optimiser pairing, the weight average — is inherited unchanged, which is the point.

12 Results

We train 8000 steps on 26.4 h of a single DJ set and merge the resulting delta onto Released. Evaluation outpaints

Model	Sampler	Distance	Level	Spread
Base, stock	50 step	0.5547	$1.10 \times$	0.11
Base + Δ	50 step	0.3817	$1.15 \times$	0.16
Released, stock	8 step	0.4537	$1.03 \times$	0.24
Released + Δ	8 step	0.2978	$1.11 \times$	0.09
the recording itself	—	—	$1.04 \times$	0.33

Table 6: Mean absolute difference between the generated region’s 32-band log-mel envelope and the true continuation’s, and the ratio of generated to true RMS. $n = 8$. The merged model is 34% closer than the released weights at the same eight-step cost, winning 7 of 8 paired renders.

160 s from a 30 s seed at four offsets drawn from the energy gate, two noise seeds each, with both models placed on identical trajectories — ping-pong redraws noise inside its own loop, so seeding the initial latent alone is not sufficient.

Two results in Table 6 are worth separating. The first is the expected one: the finetune moves the model toward the target domain. The second is that the merged eight-step model beats the fifty-step model it was trained as, 0.2978 against 0.3817. Post-trained weights sampled in eight steps are simply a better generator than base weights sampled in fifty, and the transplant makes that available to a finetune trained under the only objective the base weights admit.

The spread column is the practical finding. The released weights produced one render at $0.47 \times$ level, a partial collapse; the merged model’s worst case is $0.96 \times$, with a third of the variance. Adding a domain finetune to an adversarially post-trained model made it not only closer on average but markedly more reliable.

Stopping. The run was stopped at 8000 steps because the same evaluation at 4000 steps scored 0.3660 — inside the noise of, and marginally ahead of, 8000. The first 2000 steps bought three times what the last 2000 did. Validation loss was still falling when the run stopped, which is precisely why it is the wrong instrument to stop on; this is the same lesson as §4, arrived at from the benign direction.

13 Discussion

The result we would most want carried forward is the diagnostic. Whenever a sampler rebuilds its state from a one-step estimate — ping-pong, restart sampling, consistency-model sampling — that estimate has to be a sample and not a conditional mean, and the two are distinguishable in a single forward pass by comparing the estimate’s scale against its correlation with the truth. They coincide for a mean predictor and separate for a sample predictor. No scalar in a training log carries this:

in our runs both loss curves descended smoothly throughout, including across an interval in which the property was being destroyed.

The second result is that the division of labour between the two published checkpoints is not a compromise. The base transformer is the only one a squared-error objective is valid against; the post-trained transformer is a strictly better generator at the step count anyone actually samples at. Carrying a finetune between them as a delta gives both, and Table 6 shows the merged eight-step model beating the fifty-step model it was trained as. The margin that makes this work is finite and grows as roughly the square root of training steps, so it is worth measuring rather than assuming — one transformer layer fetched by byte range is enough.

The third is that capability is often already latent in a pretrained checkpoint, distributed across a conditioning interface trained for a broader task than the one at hand. Stable Audio 3 was trained to inpaint; outpainting is the causal corner of that distribution. Specialising to it is a change of sampling weights, not a change of model: no zero-initialised branch to warm up, no train/inference conditioning mismatch, and a step zero that is exactly the released model.

Three limitations are worth naming. The evaluation is a spectral-envelope distance on eight paired renders from a single source; it correlates with what we hear but it is not a listening study. A full finetune on one domain moves the model away from its general prior, and the residual full-mask fraction slows this without eliminating it. And the transplant margin was verified at $1.3\times$ and measured at $2.3\times$ at the reported checkpoint; longer runs will need it re-checked rather than extrapolated.

14 Conclusion

We presented a recipe for specialising a latent rectified-flow audio diffusion model to outpainting, with no architectural modification, on a single consumer GPU. The method reduces to reweighting a pretrained mask distribution, keeping the reference’s two-term objective and its orthogonalising optimiser, and choosing precisions that let 1.453 B parameters train in 24 GB.

The result we would most want carried forward is the negative one and its instrument. Squared-error training on an adversarially post-trained checkpoint is not merely suboptimal; it systematically undoes the post-training, and it does so while every scalar available during the run improves. The distinction between a sample predictor and a conditional-mean predictor is not visible in a loss curve, but it is visible in one forward pass, and it should be checked before and after any training that touches a distilled checkpoint. Code is available at <https://github.com/mrcolo/loop-training>.